

Portfolio RL Agent: Architecture Design

Trading Game for NSE/BSE Equity Markets

market-agent project

September 2026

Contents

1	Game Rules Summary	2
2	MDP Formulation	2
2.1	State Space	2
2.2	Action Space	3
2.3	Candidate Universe	3
2.4	Reward	3
2.5	Environment Transitions	3
3	Network Architecture	3
3.1	Overview	3
3.2	DualCNN Encoder (Frozen Phase)	4
3.3	Position Encoder	4
3.4	Cross-Stock Self-Attention	4
3.5	Policy Head (Actor)	4
3.6	Value Head (Critic)	5
4	Training	5
4.1	Algorithm: Soft Actor-Critic (SAC)	5
4.2	Episode Design	5
4.3	Training Phases	5
4.4	Replay Buffer	5
5	Package Structure	5
5.1	Package Dependency Rules	6
6	Pipeline Integration	6
7	Related Work	6
8	Open Questions	7

1 Game Rules Summary

The trading game defines the environment in which the RL agent operates. The rules constrain agent behaviour rather than being learned objectives.

1. The player starts with a configurable amount of cash.
2. Initial portfolio value equals the initial cash amount.
3. The player may buy and sell equities on BSE or NSE.
4. **T+2 settlement.** Proceeds from a sale move into *reserved cash*, which has the same monetary value as cash but cannot be used to fund purchases. After a 2-trading-day delay, reserved cash converts to spendable cash.
5. Purchases are capped by available (spendable) cash.
6. $V_{\text{portfolio}} = V_{\text{stocks}} + C_{\text{cash}} + C_{\text{reserved}}$.
7. **Decision frequency.** Buy/sell decisions may only be made when exchanges are open, at most once every 15 minutes, or immediately upon the release of a news item.
8. **Maximum holding period.** Any position held for more than 10 calendar days is force-sold by the environment at the prevailing market price.
9. **Objective.** Continuously maximise total portfolio value.

2 MDP Formulation

2.1 State Space

At each decision step t the agent observes a state s_t composed of four parts.

Per-stock market embedding $\mathbf{m}_i$. The pre-trained DualCNN maps recent daily and hourly OHLCV history for stock i to a 256-dimensional embedding. During the first training phase this encoder is frozen; a second fine-tuning phase can update it end-to-end.

Per-stock position state $\mathbf{p}_i$. A 4-dimensional vector describing the agent’s current exposure to stock i :

$$\mathbf{p}_i = (w_i^{\text{held}}, d_i/10, \text{pnl}_i, p_i^{\text{entry}}/p_i^{\text{current}})$$

where w_i^{held} is the current portfolio weight, $d_i \in [0, 10]$ is the number of days held (normalised by the 10-day cap), pnl_i is the unrealised return, and $p_i^{\text{entry}}/p_i^{\text{current}}$ encodes whether the position is in profit or loss.

Per-stock news signal $\mathbf{n}_i$. A 3-dimensional vector of the most recent LLM-classified signal for stock i from the NewsMonitor pipeline: (sentiment, severity, hours_since_release).

Global portfolio state $\mathbf{g}$. A 3-dimensional vector:

$$\mathbf{g} = (C_{\text{cash}}/V_{\text{portfolio}}, C_{\text{reserved}}/V_{\text{portfolio}}, \tau_{\text{reserved}}/2)$$

where τ_{reserved} is the normalised time until the oldest reserved cash tranche unlocks.

2.2 Action Space

The action is a target portfolio weight vector $\mathbf{w} \in \Delta^{N+1}$, one weight per candidate stock plus a cash weight, constrained to the probability simplex:

$$w_i \geq 0, \quad \sum_{i=0}^N w_i = 1.$$

w_0 denotes the cash allocation; $w_1, \dots, w_N$ denote allocations to the N candidate stocks. The simplex constraint is enforced by a masked softmax: any stock with $d_i \geq 10$ has its logit set to $-\infty$ before the softmax, making forced sells automatic and differentiable. This formulation follows the continuous-weight portfolio action space introduced in [5].

2.3 Candidate Universe

At each step the full 5,000+ stock universe is pre-filtered to the top $N = 100$ stocks by current SwingPredictor score. This keeps the action space fixed-size and computationally tractable while still allowing the agent to express meaningful allocation choices.

2.4 Reward

The reward at step t is the log portfolio return penalised for turnover [1]:

$$r_t = \log \frac{V_{t+1}}{V_t} - \lambda \sum_{i=1}^N |w_i^{(t)} - w_i^{(t-1)}|$$

where λ is a transaction cost coefficient calibrated to Kite brokerage rates. Using log returns makes the reward scale-invariant and additive over an episode.

2.5 Environment Transitions

The environment enforces the following mechanics on every step, in order:

- i. **Forced sells.** Any stock with $d_i \geq 10$ is sold at the opening price of the current bar; proceeds go to reserved cash.
- ii. **T+2 unlock.** Reserved cash tranches whose 2-day delay has elapsed are moved to spendable cash.
- iii. **Trade execution.** The delta between target weights $\mathbf{w}^{(t)}$ and current weights is computed. Sells execute first (freeing cash), then buys, all at the mid-price of the current 15-min bar. Sale proceeds go to reserved cash.
- iv. **Portfolio valuation.** V_{t+1} is computed from the closing price of the bar using Rule 6.
- v. **Reward computation.** r_t is calculated and the new state s_{t+1} is assembled.

3 Network Architecture

3.1 Overview

The architecture has four stages: feature extraction, per-stock encoding, cross-stock attention, and action/value heads. Figure 1 shows the data flow.

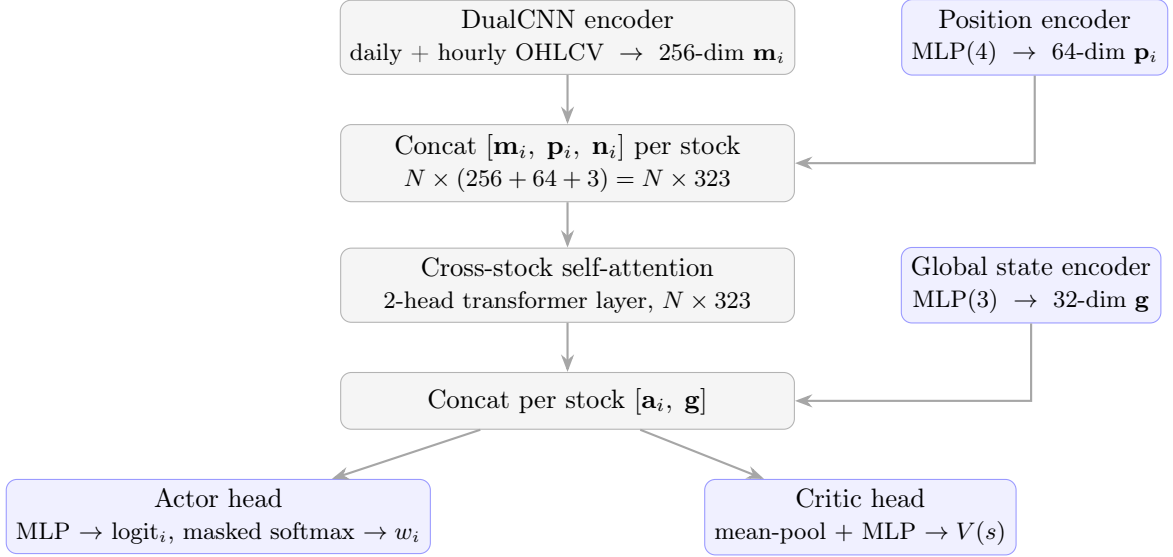


Figure 1: Data flow through the policy and value networks. $\mathbf{a}_i$ denotes the contextualised per-stock representation output by the self-attention layer.

3.2 DualCNN Encoder (Frozen Phase)

The existing `DualCNN_v3` from `StockSwingPredictor` produces a 256-dimensional market embedding $\mathbf{m}_i$ per stock from a 28-day daily window and a 280-bar hourly window. Its weights are frozen during initial RL training to avoid destroying the pre-trained price representations.

3.3 Position Encoder

A two-layer MLP maps the 4-dimensional position state $\mathbf{p}_i$ to a 64-dimensional embedding. This gives the agent an explicit representation of its current exposure to each stock, decoupled from the market signal.

3.4 Cross-Stock Self-Attention

After concatenating $[\mathbf{m}_i, \mathbf{p}_i, \mathbf{n}_i]$ for all N candidate stocks, a single transformer encoder layer [13] with 2 attention heads produces contextualised per-stock representations. This design is directly motivated by Relational Deep Reinforcement Learning [14], where self-attention allows an RL agent to reason about relationships between entities in its observation — here, the relative opportunity between stocks. It allows the agent to reason about relative opportunities, e.g. rotating from a stock with high unrealised gains into one with a stronger forward signal.

3.5 Policy Head (Actor)

For each stock i , the contextualised representation is concatenated with the global state embedding $\mathbf{g}$ and passed through a small MLP to produce a scalar logit l_i . A special cash logit l_0 is produced from $\mathbf{g}$ alone. The target weights are:

$$w_i = \frac{\exp(l_i) \cdot \mathbf{1}[d_i < 10]}{\sum_j \exp(l_j) \cdot \mathbf{1}[d_j < 10]}$$

The mask $\mathbf{1}[d_i < 10]$ zeroes out stocks subject to forced sell before normalisation, so the constraint from Rule 8 is satisfied by construction.

3.6 Value Head (Critic)

The critic mean-pools the contextualised stock representations, concatenates the global state $\mathbf{g}$, and passes the result through an MLP to produce a scalar state value $V(s)$.

4 Training

4.1 Algorithm: Soft Actor-Critic (SAC)

SAC [3, 4] is chosen for three reasons:

- **Off-policy.** A replay buffer allows multiple gradient updates per environment step, making historical data reuse efficient.
- **Continuous actions.** SAC handles continuous action spaces natively via reparameterisation.
- **Entropy regularisation.** The maximum entropy objective encourages portfolio diversification — the agent is penalised for collapsing all weight onto a single stock.

The SAC objective for the actor is:

$$\mathcal{L}_\pi = \mathbb{E}_{s \sim \mathcal{B}} \left[\alpha \log \pi(\mathbf{w} \mid s) - Q(s, \mathbf{w}) \right]$$

where α is a learned entropy temperature (tuned automatically [4]) and $\mathcal{B}$ is the replay buffer. PPO [10] is retained as an on-policy baseline for comparison.

4.2 Episode Design

Each training episode is a randomly sampled 6-month window from the historical dataset (2010–2025), comprising approximately 6,000 decision steps at 15-minute resolution. The environment is initialised with a configurable starting cash value. Multiple episodes run in parallel on the historical replay.

4.3 Training Phases

Phase 1 — Frozen encoder. DualCNN weights are fixed. Only the position encoder, attention layer, and actor/critic heads are trained. This is faster per step and prevents destroying the pre-trained market representations early in training.

Phase 2 — End-to-end fine-tuning. All weights are unfrozen with a lower learning rate on the DualCNN layers. The policy signal can now shape the encoder towards portfolio-relevant features rather than swing-prediction features. Adversarial training [6] may be added in this phase to improve robustness to distribution shift between historical training and live deployment.

4.4 Replay Buffer

A circular buffer stores transitions $(s_t, \mathbf{w}_t, r_t, s_{t+1}, \text{done}_t)$. Because the environment is a deterministic historical replay, experience generation is fast and the buffer can be seeded from many parallel episode workers before training begins.

5 Package Structure

A new `TradingGame` package is added under `packages/`. It depends on `StockSwingPredictor` (for the DualCNN and OHLCV cache) and `NewsMonitor` (for news signals).

```

packages/TradingGame/
+-- Project.toml
+-- src/
    +-- TradingGame.jl # module entry, exports
    +-- types.jl # Portfolio, TradingState, TradingAction,
      | # ReservedCashTranche, Episode, Transition
    +-- environment.jl # step!, reset!, forced-sell, T+2 accounting
    +-- features.jl # assemble state from InferenceCache + news
    +-- policy.jl # position encoder, attention, actor/critic heads
    +-- sac.jl # SAC update (actor loss, critic loss, alpha loss)
    +-- replay_buffer.jl # circular buffer of Transition
    +-- backtest.jl # run episode on historical data, return equity curve
    +-- display.jl # Base.show overrides for Portfolio, Episode

```

5.1 Package Dependency Rules

Package	Dependencies
TradingGame	StockSwingPredictor, NewsMonitor, Flux.jl
StockSwingPredictor	TijoriData
NewsMonitor	(none within the project)

6 Pipeline Integration

The full data-to-decision pipeline, integrating the new package with the existing infrastructure, is as follows:

```

# Morning setup
node sidecar/kite_login.js # fresh Kite session
julia scripts/update_ohlc.jl --include-bse # append yesterday's bars
julia scripts/build_cache.jl # rebuild InferenceCache

# Training (offline, run once / periodically)
julia --project=packages/TradingGame scripts/train_rl_agent.jl

# Live inference loop (during market hours)
julia --project=packages/TradingGame scripts/run_agent.jl
-> every 15 min: assemble state from InferenceCache + live prices
-> or on news: NewsMonitor fires, agent re-evaluates immediately
-> policy outputs target weights
-> environment executes delta trades via Kite Connect orders API

```

7 Related Work

Portfolio management with RL. The most direct precedent for this architecture is Jiang et al. [5], who frame portfolio management as an MDP with a continuous simplex-constrained action space — the same formulation used here. They apply an Exponential Gradient update and demonstrate that learning directly over portfolio weights outperforms single-asset trading agents. Liang et al. [6] extend this framework with adversarial training to improve robustness to distribution shift between training and deployment, which is directly relevant to the gap between our historical backtesting environment and live market conditions. Liu et al. [7] provide a comprehensive library and benchmarking framework for portfolio RL that is useful for evaluating baselines. The foundational objective of learning a trading policy via direct reinforcement

learning was established by Moody and Saffell [9], whose recurrent formulation underpins most subsequent work. Deng et al. [2] combine CNNs for feature extraction with an RL trading objective, foreshadowing the frozen-encoder approach taken here.

Attention in RL. The cross-stock self-attention layer is motivated by Relational Deep Reinforcement Learning [14] (DeepMind), which introduces multi-head self-attention inside an RL policy to allow the agent to reason about pairwise relationships between entities in its observation. The underlying attention mechanism is the scaled dot-product attention of Vaswani et al. [13].

SAC. The training algorithm is Soft Actor-Critic [3], an off-policy maximum-entropy actor-critic method for continuous action spaces. The version with automatic entropy temperature tuning is used [4]. PPO [10] is included as an on-policy baseline.

Transaction costs. The turnover penalty in the reward function follows the optimal execution framework of Almgren and Chriss [1], the canonical model for market impact and transaction cost penalties in portfolio optimisation.

Deep RL for games. The broader context of applying deep networks as function approximators in game-like sequential decision problems was established by DQN [8], then extended to superhuman performance in Go by AlphaGo [11] and generalised across board games by AlphaZero [12]. While these systems address discrete-action, perfect-information settings, they demonstrate the scalability of the deep RL paradigm that underpins this architecture.

8 Open Questions

1. **Candidate universe size N .** $N = 100$ is the current proposal. A larger N (e.g. 500) increases action space expressiveness but scales the attention layer as $O(N^2)$. This is the primary remaining architectural decision.
2. **Transaction cost model.** Kite brokerage is flat Rs.20 per executed order for equity delivery. The turnover penalty λ should be calibrated to reflect this in normalised return terms.
3. **Dirichlet vs. softmax-Gaussian policy.** A Dirichlet distribution is the natural choice for simplex-constrained outputs and may produce better-calibrated uncertainty. Softmax of Gaussian logits is simpler to implement. Worth benchmarking both.
4. **Order placement.** The `broker.jl` module currently reads portfolio state but does not place orders. Kite Connect order placement (market orders for live execution) needs to be added before live deployment.

References

- [1] Almgren, R. and Chriss, N. (2001). Optimal execution of portfolio transactions. *Journal of Risk*, 3(2):5–39.
- [2] Deng, Y., Bao, F., Kong, Y., Ren, Z., and Dai, Q. (2017). Deep direct reinforcement learning for financial signal representation and trading. *IEEE Transactions on Neural Networks and Learning Systems*, 28(3):653–664.
- [3] Haarnoja, T., Zhou, A., Abbeel, P., and Levine, S. (2018a). Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pages 1861–1870.

- [4] Haarnoja, T., Zhou, A., Hartikainen, K., Tucker, G., Ha, S., Tan, J., Kumar, V., Zhu, H., Gupta, A., Abbeel, P., and Levine, S. (2018b). Soft actor-critic algorithms and applications. *arXiv preprint arXiv:1812.05905*.
- [5] Jiang, Z., Xu, D., and Liang, J. (2017). A deep reinforcement learning framework for the financial portfolio management problem. *arXiv preprint arXiv:1706.10059*.
- [6] Liang, Z., Chen, H., Zhu, J., Jiang, K., and Li, Y. (2018). Adversarial deep reinforcement learning in portfolio management. *arXiv preprint arXiv:1808.09940*.
- [7] Liu, X., Rui, J., Gao, J., Yang, L., Yang, H., Zhu, M., Wang, C., Wang, Z., and Guo, J. (2021). FinRL: A deep reinforcement learning library for automated stock trading in quantitative finance. In *Proceedings of the Second ACM International Conference on AI in Finance (ICAIF)*.
- [8] Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., and Hassabis, D. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533.
- [9] Moody, J. and Saffell, M. (2001). Learning to trade via direct reinforcement. *IEEE Transactions on Neural Networks*, 12(4):875–889.
- [10] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. (2017). Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*.
- [11] Silver, D., Huang, A., Maddison, C. J., Guez, A., Sifre, L., van den Driessche, G., Schrittwieser, J., Antonoglou, I., Panneershelvam, V., Lanctot, M., Dieleman, S., Grewe, D., Nham, J., Kalchbrenner, N., Sutskever, I., Lillicrap, T., Leach, M., Kavukcuoglu, K., Graepel, T., and Hassabis, D. (2016). Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587):484–489.
- [12] Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., Lanctot, M., Sifre, L., Kumaran, D., Graepel, T., Lillicrap, T., Simonyan, K., and Hassabis, D. (2018). A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362(6419):1140–1144.
- [13] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30.
- [14] Zambaldi, V., Raposo, D., Santoro, A., Bapst, V., Li, Y., Babuschkin, I., Tuyls, K., Reichert, D., Lillicrap, T., Lockhart, E., Shanahan, M., Langston, V., Pascanu, R., Botvinick, M., Vinyals, O., and Battaglia, P. (2018). Relational deep reinforcement learning. *arXiv preprint arXiv:1806.01830*.